

API SECURITY RISK ANALYSIS REPORT

Future Interns Fellowship Program — Cyber Security Internship

Task 03 | May 2026

Prepared by: AARISHA R.

Report Detail	Information
APIs Tested	JSONPlaceholder (jsonplaceholder.typicode.com) ReqRes (reqres.in)
Testing Tool	Postman
Test Type	Read-Only Security Analysis (Ethical — No Exploitation)
Total Findings	6 Security Risks Identified
Critical Findings	1 Critical 3 High 1 Medium 1 Informational
Report Date	May 2026
Scope	Public Demo APIs — Safe for Educational Security Testing

1. Executive Summary

This report presents the findings of a read-only API Security Risk Analysis conducted on two publicly available demo APIs: JSONPlaceholder and ReqRes. The analysis was performed as part of the Future Interns Cyber Security Internship Fellowship Program (Task 03).

The assessment identified 6 security risks across the tested APIs, including 1 Critical vulnerability, 3 High severity issues, 1 Medium severity issue, and 1 Informational finding. The most serious finding was the ability to perform destructive DELETE operations on API resources without any authentication — a real-world vulnerability that could allow attackers to destroy data in production systems.

These findings are consistent with the OWASP API Security Top 10, the industry standard framework for API security risk assessment. All testing was conducted ethically using only read-only and safe operations on demo APIs designed specifically for educational testing.

Risk Level	Count	Description
 CRITICAL	1	Unauthenticated DELETE — destructive action with no auth required
 HIGH	3	Unauthenticated access, excessive data exposure, IDOR vulnerability
 MEDIUM	1	Missing rate limiting — potential for abuse/DoS
 INFORMATIONAL	1	ReqRes now enforces API key authentication (good practice noted)

2. Scope & Methodology

2.1 APIs Tested

API	Base URL	Purpose
JSONPlaceholder	https://jsonplaceholder.typicode.com	Free fake REST API for testing and prototyping
ReqRes	https://reqres.in	Hosted REST API for testing front-end applications

2.2 Tools Used

Tool	Version / Type	Purpose
Postman	Desktop Application	API endpoint testing and response inspection
Browser DevTools	Chrome / Edge	Header inspection and network analysis
OWASP API Security Top 10	2023 Edition	Security risk classification framework

2.3 Testing Methodology

The following steps were performed for each API endpoint:

- Step 1 — Endpoint Discovery: Identified all available API endpoints from documentation
- Step 2 — Authentication Testing: Tested each endpoint without authentication credentials
- Step 3 — Response Analysis: Inspected response data for sensitive information exposure
- Step 4 — Authorization Testing: Tested access to other users' resources by modifying IDs
- Step 5 — HTTP Method Testing: Tested destructive methods (DELETE) without authentication
- Step 6 — Rate Limit Testing: Sent multiple rapid requests to check for rate limiting
- Step 7 — Risk Classification: Classified each finding using OWASP API Security Top 10

2.4 Ethical Boundaries

All testing was conducted within the following ethical boundaries:

- Only public demo APIs designed for testing were used
- No exploitation or bypass attempts were made
- No denial-of-service or flooding attacks were performed
- No private or production APIs were tested
- All findings are reported for educational and awareness purposes only

3. Security Findings

A total of 6 security findings were identified during the assessment. Each finding is documented below with its severity, evidence, business impact, and recommended remediation.

Finding 1: Unauthenticated Access to All Endpoints

Attribute	Detail
Severity	 HIGH
OWASP Category	API2:2023 — Broken Authentication
Endpoint Tested	GET https://jsonplaceholder.typicode.com/users
HTTP Status	200 OK
Authentication Required	None — Fully Open
Response Size	2.94 KB of user data

Description:

All API endpoints on JSONPlaceholder return full data responses with a 200 OK status code without requiring any form of authentication. Any person or automated script can access all user data, posts, comments, and other resources without providing a username, password, or API token.

Business Impact:

In a real-world production API, this vulnerability would allow anyone on the internet to freely access all customer data, personal information, and business records without needing to log in. This represents a severe data breach risk and could result in regulatory fines (GDPR, DPDP Act), reputational damage, and legal liability.

Remediation:

- Implement authentication on all API endpoints (OAuth 2.0, JWT tokens, or API keys)
- Require a valid Bearer token in the Authorization header for all requests
- Return 401 Unauthorized for all unauthenticated requests
- Implement session management with token expiry

Finding 2: Excessive Data Exposure

Attribute	Detail
Severity	 HIGH
OWASP Category	API3:2023 — Broken Object Property Level Authorization
Endpoints Tested	GET /users GET /users/1 GET /comments
HTTP Status	200 OK on all requests
Data Exposed	Full name, username, email, phone, full address, GPS coordinates, company name
Comments Response Size	40.81 KB of raw data returned

Description:

The API returns significantly more data than is necessary for any typical use case. The /users endpoint returns GPS latitude/longitude coordinates, full home addresses, phone numbers, company internal catchphrases, and business strategy descriptions alongside basic profile information. The /comments endpoint returns 40.81 KB of data in a single unfiltered response.

Business Impact:

Exposing unnecessary fields increases the attack surface and privacy risk. Sensitive fields like GPS coordinates and full addresses could be used for physical tracking or stalking. Excess data also increases bandwidth costs and slows application performance. Under privacy regulations like GDPR, collecting and exposing more data than necessary is a compliance violation.

Remediation:

- Apply the principle of data minimisation — only return fields the client actually needs
- Implement response filtering to strip sensitive fields before sending
- Use API response schemas that explicitly define which fields are returned
- Implement pagination to limit the volume of data returned per request
- Conduct a data classification review to identify which fields are sensitive

Finding 3: Insecure Direct Object Reference (IDOR)

Attribute	Detail
Severity	 HIGH
OWASP Category	API1:2023 — Broken Object Level Authorization (BOLA)
Endpoint Tested	GET https://jsonplaceholder.typicode.com/users/1
HTTP Status	200 OK
Attack Vector	Change user ID in URL from /users/1 to /users/2, /users/3 etc.
Result	Full profile of any user accessible by incrementing the ID

Description:

The API uses sequential numeric IDs to identify user resources (e.g., /users/1, /users/2). There is no authorization check to verify whether the requesting user has permission to access the requested resource. By simply changing the number in the URL, an attacker can access the complete profile of any user in the system. This is known as an Insecure Direct Object Reference (IDOR) or Broken Object Level Authorization (BOLA) vulnerability.

Business Impact:

In a production system, this vulnerability would allow any authenticated (or even unauthenticated) user to view the personal data of every other user in the database. This is one of the most common causes of large-scale data breaches. It could expose personally identifiable information (PII), financial records, medical data, or other sensitive user information to unauthorized parties.

Remediation:

- Implement object-level authorization checks on every API request
- Verify that the authenticated user has permission to access the requested resource ID
- Use random UUIDs instead of sequential integers as resource identifiers
- Never trust user-supplied IDs without server-side authorization verification
- Log and alert on attempts to access resources belonging to other users

Finding 4: Unauthenticated DELETE Operation [CRITICAL]

Attribute	Detail
Severity	 CRITICAL
OWASP Category	API2:2023 — Broken Authentication + API5:2023 — Broken Function Level Authorization
Endpoint Tested	DELETE https://jsonplaceholder.typicode.com/posts/1
HTTP Status	200 OK — DELETE SUCCEEDED
Authentication Required	None — No token, no login, no verification
Result	Resource deleted successfully without any credentials

Description:

A DELETE request was sent to the /posts/1 endpoint without any authentication credentials, headers, or tokens. The API returned a 200 OK response, confirming that the delete operation was accepted and processed. This means any person or automated script can permanently delete any resource in the system without logging in or providing any proof of identity or ownership.

Business Impact:

This is the most severe finding in this assessment. In a real-world production API, this vulnerability would allow any attacker to delete any data in the system — including customer records, transaction histories, user accounts, or critical business data. A single automated script could systematically destroy an entire database. This could result in complete data loss, service outage, financial loss, regulatory penalties, and permanent reputational damage. Recovery would require full database restoration from backups, if backups exist.

Remediation:

- IMMEDIATELY require authentication for all write operations (POST, PUT, PATCH, DELETE)
- Implement role-based access control (RBAC) — only resource owners can delete their data
- Require re-authentication or confirmation for destructive operations
- Implement audit logging for all DELETE requests with user identity and timestamp
- Add soft-delete functionality (mark as deleted) instead of permanent deletion
- Set up real-time alerts for unusual volumes of DELETE requests

Finding 5: Missing Rate Limiting

Attribute	Detail
Severity	● MEDIUM
OWASP Category	API4:2023 — Unrestricted Resource Consumption
Endpoints Tested	All GET endpoints on JSONPlaceholder
Test Method	Multiple rapid successive requests
Result	No rate limiting detected — unlimited requests accepted
Risk	Susceptible to automated abuse, scraping, and DoS attacks

Description:

No rate limiting mechanism was detected on the tested API endpoints. Multiple rapid successive requests were made without receiving any 429 Too Many Requests response or throttling. This means an attacker or automated script could send unlimited requests to the API without restriction.

Business Impact:

Without rate limiting, the API is vulnerable to automated data scraping (extracting all user data in seconds), brute force attacks (testing thousands of IDs), and denial-of-service attacks (overwhelming the server with requests). This can lead to server performance degradation, increased infrastructure costs, complete service unavailability, and mass data theft.

Remediation:

- Implement rate limiting at the API gateway level (e.g., 100 requests per minute per IP)
- Return 429 Too Many Requests with Retry-After header when limits are exceeded
- Implement exponential backoff for repeated violations
- Use IP-based and user-based rate limiting in combination
- Monitor and alert on unusual request volume patterns

Finding 6: ReqRes API Key Enforcement [Informational]

Attribute	Detail
Severity	● INFORMATIONAL (Positive Finding)
OWASP Category	API2:2023 — Broken Authentication (Correctly Implemented)
Endpoint Tested	GET https://reqres.in/api/users/1
HTTP Status	401 Unauthorized

Attribute	Detail
Authentication Required	Yes — x-api-key header required
Result	API correctly rejected unauthenticated request

Description:

Unlike JSONPlaceholder, the ReqRes API has implemented API key authentication. When a request was made without the required x-api-key header, the API correctly returned a 401 Unauthorized response with a clear error message explaining the requirement. The error response also included helpful guidance on how to obtain a valid API key.

Business Impact:

This is a positive security control. By requiring API key authentication, ReqRes prevents unauthorized access to its endpoints and can track and control API usage per client. This is the correct behaviour that all production APIs should implement.

Recommendation:

- This is a good security practice — all APIs should implement similar authentication controls
- JSONPlaceholder should adopt a similar authentication model for its endpoints
- API keys should be rotated regularly and have expiry dates
- Failed authentication attempts should be logged and monitored

4. Risk Summary Dashboard

#	Finding	Endpoint	Severity	OWASP Category
1	Unauthenticated Access to All Endpoints	GET /users, /posts, /comments	 HIGH	API2 — Broken Authentication
2	Excessive Data Exposure	GET /users, /comments (40.81 KB)	 HIGH	API3 — Broken Object Property Auth
3	Insecure Direct Object Reference (IDOR)	GET /users/{id}	 HIGH	API1 — Broken Object Level Auth
4	Unauthenticated DELETE Operation	DELETE /posts/1 → 200 OK	 CRITICAL	API2 + API5 — Broken Auth + Function Auth
5	Missing Rate Limiting	All Endpoints	 MEDIUM	API4 — Unrestricted Resource Consumption
6	ReqRes API Key Enforcement	GET /api/users/1 → 401	 INFO	API2 — Authentication (Correct Implementation)

Overall Risk Rating: HIGH

The presence of a CRITICAL finding (unauthenticated DELETE) combined with 3 HIGH severity findings results in an overall HIGH risk rating for the JSONPlaceholder API. Immediate remediation of authentication controls is strongly recommended before any similar API pattern is used in production.

5. Business Impact Summary

The following table summarises the potential real-world business impact if these vulnerabilities existed in a production API:

Risk Area	Potential Impact	Likelihood
Data Breach	All user PII exposed to any internet user — GDPR/DPDP violation	● Very High
Data Destruction	Entire database wiped by unauthenticated DELETE requests	● Very High
Financial Loss	Regulatory fines, incident response costs, service downtime revenue loss	● High
Reputation Damage	Customer trust lost permanently after public data breach disclosure	● High
Service Disruption	API flooded with requests causing downtime for legitimate users	● Medium
Competitive Harm	Business data scraped and used by competitors	● Medium

6. Remediation Roadmap

Priority	Action	Timeline	Findings Addressed
1 — IMMEDIATE	Implement authentication (JWT/OAuth) on all endpoints	Week 1	Finding 1, 4
2 — IMMEDIATE	Require auth for all write operations (POST, PUT, DELETE)	Week 1	Finding 4
3 — SHORT-TERM	Implement object-level authorization checks (BOLA fix)	Week 2	Finding 3
4 — SHORT-TERM	Apply data minimisation — strip unnecessary response fields	Week 2	Finding 2
5 — MEDIUM-TERM	Implement rate limiting at API gateway level	Week 3-4	Finding 5
6 — ONGOING	Regular API security audits and penetration testing	Quarterly	All Findings

Home Workspaces API Network

Search Postman

Home

Overview Getting started **API** <https://jsonplaceholder.typicode.com/users> [Save](#) [Share](#)

GET <https://jsonplaceholder.typicode.com/users> [Send](#)

Auth: Basic

Headers (0)

Body Scripts Tests Settings

Query Params

Key Value Description

Body

JSON

```
1 {
2   "id": 1,
3   "name": "Leanne Graham",
4   "username": "Bret",
5   "email": "Sincere@april.biz",
6   "address": {
7     "street": "Kulas Light",
8     "suite": "Apt. 556",
9     "city": "Newburgh",
10    "zipcode": "10994-3039",
11    "geo": {
12      "lat": "37.3159",
13      "lng": "-78.0421"
14    }
15  },
16  "phone": "570-755-8030 x3400",
17  "website": "hildegard.org",
18  "company": {
19    "name": "Romaguera-Corp",
20    "catchphrase": "Multi-layered client-ready neural-net",
21    "bs": "harness real-time e-markets"
22  }
23 }
```

100 OK 1.57 s 1.94 KB

Runner Start Proxy Cookies View Team

Home Workspaces API Network

Search Postman

Home

Overview Getting started **API** <https://jsonplaceholder.typicode.com/users/1> [Save](#) [Share](#)

GET <https://jsonplaceholder.typicode.com/users/1> [Send](#)

Auth: Basic

Headers (0)

Body Scripts Tests Settings

Query Params

Key Value Description

Body

JSON

```
1 {
2   "id": 1,
3   "name": "Leanne Graham",
4   "username": "Bret",
5   "email": "Sincere@april.biz",
6   "address": {
7     "street": "Kulas Light",
8     "suite": "Apt. 556",
9     "city": "Newburgh",
10    "zipcode": "10994-3039",
11    "geo": {
12      "lat": "37.3159",
13      "lng": "-78.0421"
14    }
15  },
16  "phone": "570-755-8030 x3400",
17  "website": "hildegard.org",
18  "company": {
19    "name": "Romaguera-Corp",
20    "catchphrase": "Multi-layered client-ready neural-net",
21    "bs": "harness real-time e-markets"
22  }
23 }
```

100 OK 2.05 s 1.44 KB

Runner Start Proxy Cookies View Team

7. Conclusion

This API Security Risk Analysis identified 6 security findings across two public demo APIs, with the most critical being the ability to perform unauthenticated DELETE operations — a vulnerability that could lead to complete data destruction in a production environment.

The findings align closely with the OWASP API Security Top 10, confirming that the most common and dangerous API vulnerabilities — broken authentication, excessive data exposure, and broken object level authorization — remain prevalent in modern API design.

The key lesson from this assessment is that APIs must never be designed with a 'security later' mindset. Authentication, authorization, data minimisation, and rate limiting must be built into the API from the very beginning. Every endpoint, every HTTP method, and every response field must be deliberately designed with security in mind.

For organisations deploying APIs in production, it is strongly recommended to conduct regular API security audits, implement the OWASP API Security Top 10 controls, and train development teams on secure API design principles.

8. References

- OWASP API Security Top 10 (2023): <https://github.com/OWASP/API-Security>
- API Security Checklist: <https://github.com/shieldfy/API-Security-Checklist>
- JSONPlaceholder Test API: <https://jsonplaceholder.typicode.com>
- ReqRes API Testing Platform: <https://reqres.in>
- Postman API Testing Tool: <https://www.postman.com>
- Future Interns Fellowship Program — Cyber Security Internship, Task 03